UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CONTADURÍA Y ADMINISTRACIÓN

SISTEMA DE UNIVERSIDAD ABIERTA

Alumno: Emanuel Marzano Villalobos

Programación De Dispositivos Móviles

Cristian Cardoso Arellano

Grupo 9697

m3_01

09/10/2026

Enlace a repositorio de la actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M3_01

1. Introducción

Una calculadora parece un programa sencillo, pero es un buen ejercicio para aprender a organizar el código. En lugar de escribir todas las instrucciones en un solo lugar, la programación orientada a objetos permite dividir el programa en piezas pequeñas, cada una con una sola responsabilidad, que se comunican entre sí.

En esta actividad se desarrolla una calculadora en lenguaje Java dentro de Android Studio, sin interfaz gráfica, que realiza suma, resta, multiplicación y división, y que valida los resultados inválidos (NaN o infinito) que se producen al dividir entre cero. Para ello se parte de las clases base proporcionadas por el asesor, que definen interfaces y enumeraciones, y se crean las clases que las implementan. Los resultados se imprimen en la ventana Logcat.

2. Objetivo

Desarrollar una calculadora sin interfaz gráfica en lenguaje Java que realice las operaciones de suma, resta, multiplicación y división, aplicando los conceptos de interfaces, enumeraciones, constantes, abstracción y modularización, e incluyendo la validación de resultados no numéricos (NaN).

3. Conceptos utilizados

Antes de presentar el código, conviene explicar los conceptos de Java en los que se basa la actividad:

- **`Clase`**: es el molde a partir del cual se crean objetos. Define qué datos guarda (variables) y qué puede hacer (métodos).
- **`Interfaz (interface)`**: es un contrato: indica qué métodos debe tener una clase, pero no cómo funcionan. Sus métodos no tienen código, solo terminan en punto y coma.
- **`implements`**: palabra que indica que una clase cumple el contrato de una interfaz y, por lo tanto, está obligada a escribir todos sus métodos.
- **`@Override`**: anotación que señala que un método viene de una interfaz (o de una clase padre) y se está escribiendo su funcionamiento.
- **`Enumeración (enum)`**: es una lista fija de opciones con nombre. Por ejemplo, las cuatro operaciones posibles de la calculadora.
- **`Constante (final)`**: variable cuyo valor no puede cambiar una vez asignado. Por convención se escribe en mayúsculas.
- **`Constructor`**: método especial que se ejecuta al crear un objeto con new y que sirve para prepararlo.
- **`Abstracción`**: trabajar con lo que hace una pieza (su interfaz) sin depender de cómo lo hace (su clase concreta).

- **`Modularización`**: dividir el programa en módulos independientes, cada uno con una sola responsabilidad.
- **`NaN (Not a Number)`**: valor especial de los números decimales que significa “no es un número”; aparece, por ejemplo, al calcular 0 ÷ 0. Dividir otro número entre cero da Infinity (infinito).

La calculadora quedó organizada de la siguiente manera:

Clase	Interfaz que cumple	Responsabilidad
`Procesamiento`	`IProcesamiento`	Realiza las cuatro operaciones aritméticas.
`Display`	`IDisplay`	Construye los mensajes de resultado y de error.
`Control`	`IControl`	Guarda los valores y la operación, y decide qué cálculo hacer.
`Calculadora`	`ICalculadora`	Une las tres piezas y valida el resultado (NaN o infinito).
`MainActivity`	—	Crea los objetos, arma la calculadora y ejecuta las pruebas.

4. Desarrollo

4.1. Creación del proyecto

Se creó el proyecto M3_01 con la plantilla Empty Views Activity y el lenguaje Java. Todas las clases se crearon dentro del paquete com.example.m3_01, ubicado en la carpeta kotlin+java, mediante clic derecho → New → Java Class, eligiendo en cada caso el tipo correspondiente (Class, Interface o Enum).

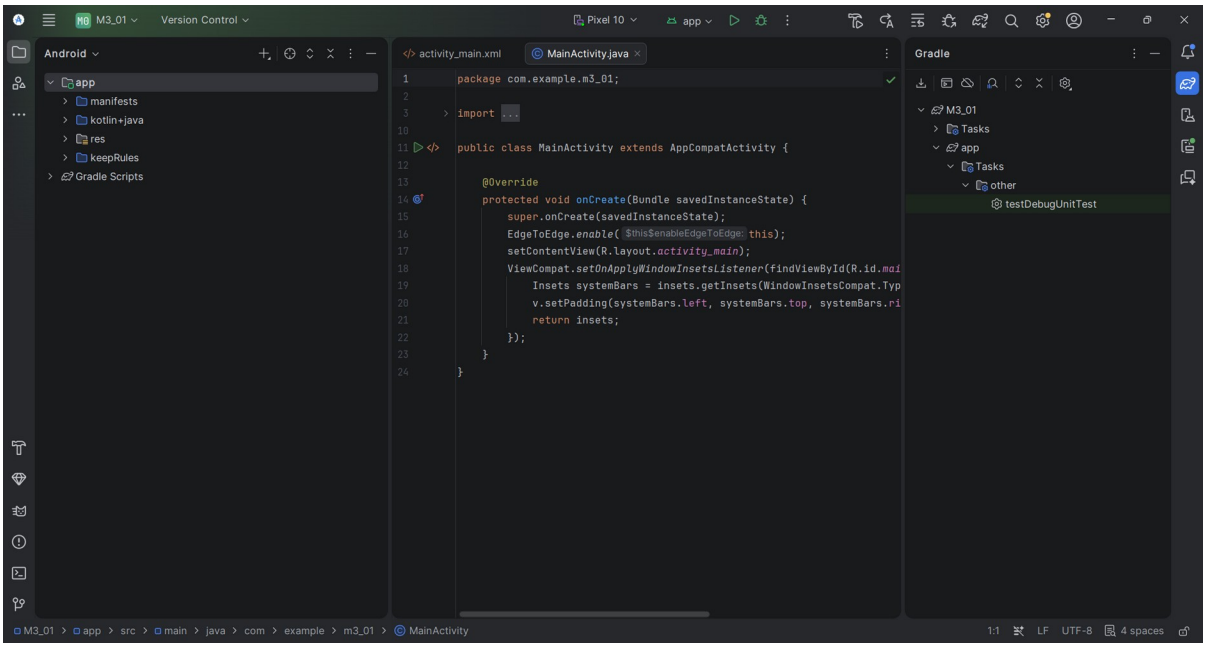


Figura 1. Proyecto M3_01 creado.

4.2. Clases base proporcionadas por el asesor

Las siguientes seis clases forman parte del material base de la actividad. Definen las reglas que deben cumplir las clases de la calculadora.

Enumeración Operacion

```
public enum Operacion {
    SUM, RES, MUL, DIV
}
```

- **`public enum Operacion`**: declara una enumeración pública llamada Operacion.
- **`SUM, RES, MUL, DIV`**: son las cuatro operaciones posibles: suma, resta, multiplicación y división. Al ser un enum, no se puede usar ninguna operación fuera de esta lista, lo que evita errores como escribir mal el nombre de una operación.

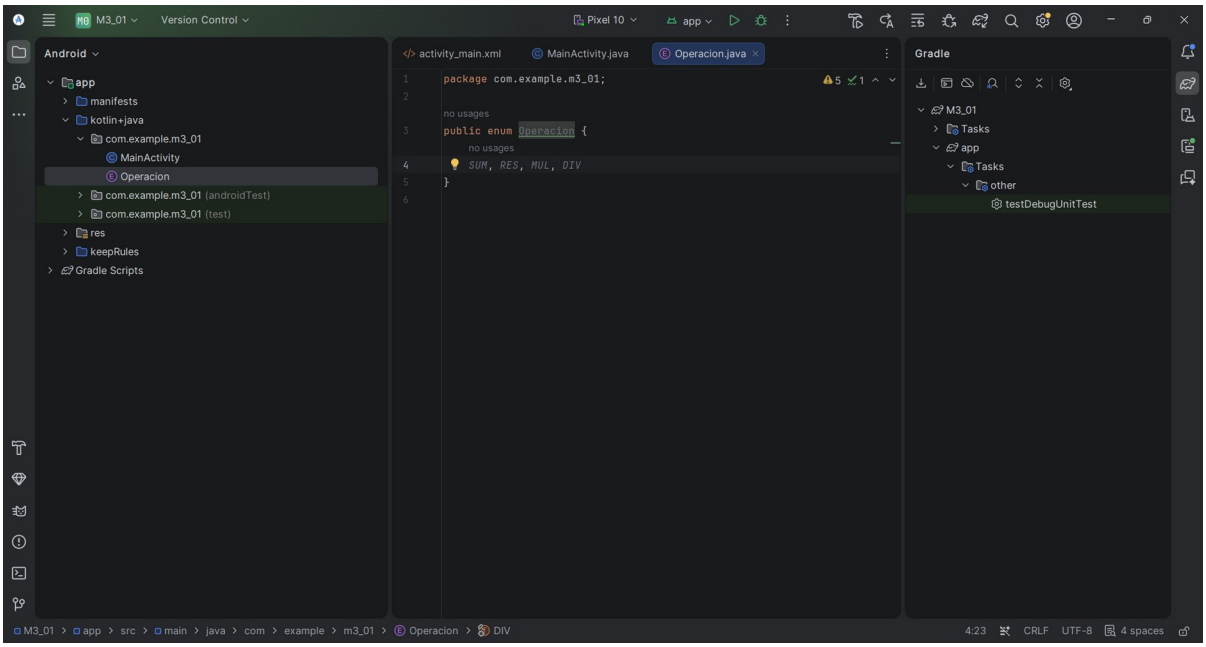


Figura 2. Enumeración Operacion.

Enumeración CalculadoraError

```
public enum CalculadoraError {
    DIV_ZERO, MAX_NUMBER, IMAGINARIO
}
```

- **`DIV_ZERO`**: error de división entre cero.
- **`MAX_NUMBER`**: error cuando el resultado supera el número máximo que se puede representar.
- **`IMAGINARIO`**: error cuando el resultado sería un número imaginario (por ejemplo, la raíz cuadrada de un negativo). Se incluye en la lista aunque esta calculadora no tiene operaciones que lo produzcan.

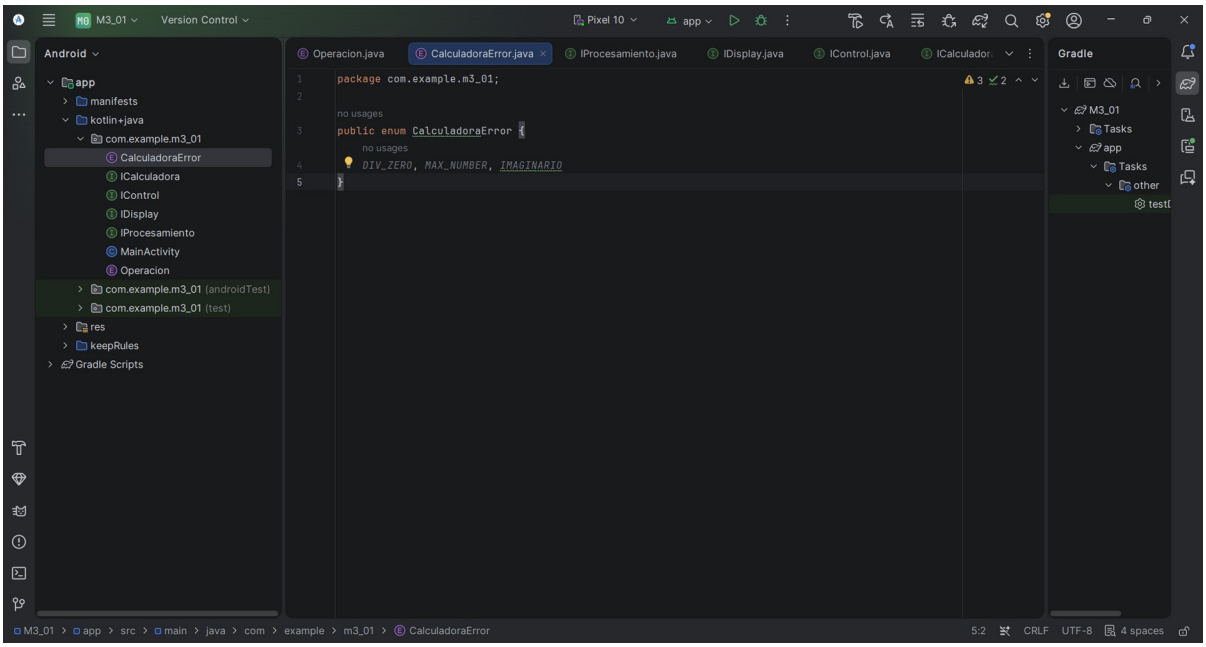


Figura 3. Enumeración CalculadoraError.

Interfaz IProcesamiento

```
public interface IProcesamiento {
    float suma(float x, float y);
    float resta(float x, float y);
    float multi(float x, float y);
    float div(float x, float y);
}
```

- **`public interface IProcesamiento`**: declara el contrato de la pieza que hace las cuentas.
- **`float suma(float x, float y);`**: cada línea indica un método que recibe dos números decimales (float) y devuelve otro. No tiene llaves ni código porque la interfaz solo dice qué debe existir; el cómo lo escribe la clase que la implemente.

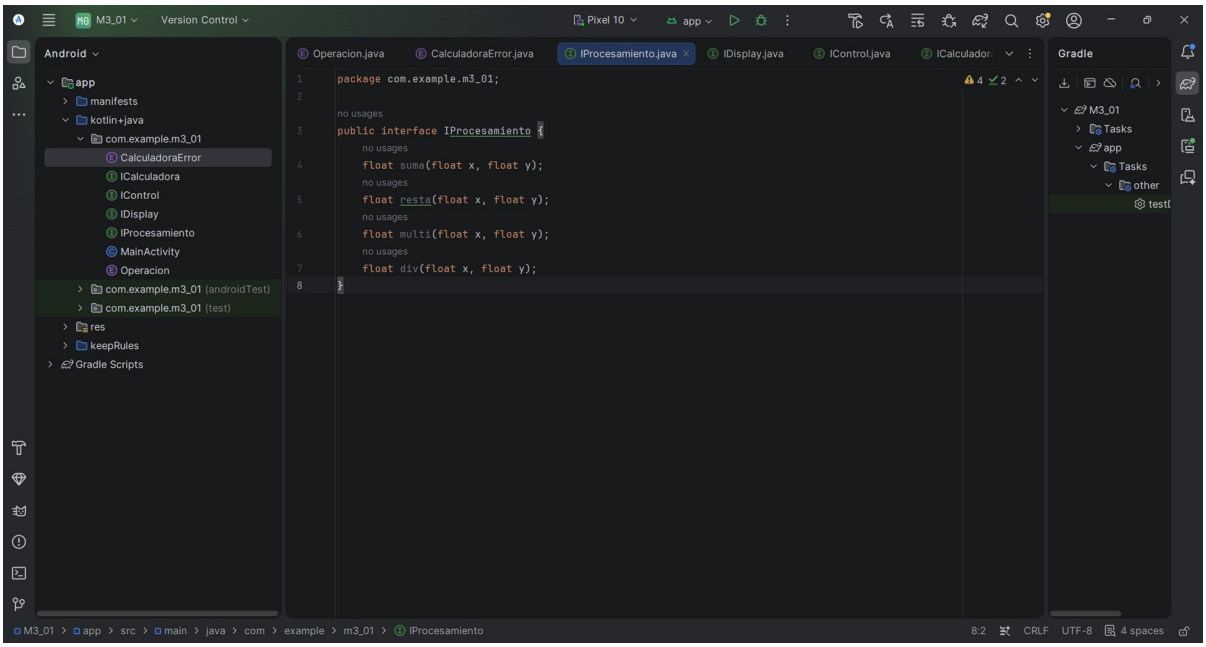


Figura 4. Interfaz IProcesamiento.

Interfaz IDisplay

```
public interface IDisplay {
    String muestraResultado(float res);
    String muestraError(CalculadoraError error);
}
```

- **`muestraResultado(float res)`**: recibe un número y debe devolver un texto (String) para mostrarlo.
- **`muestraError(CalculadoraError error)`**: recibe uno de los errores del enum y debe devolver el mensaje correspondiente.

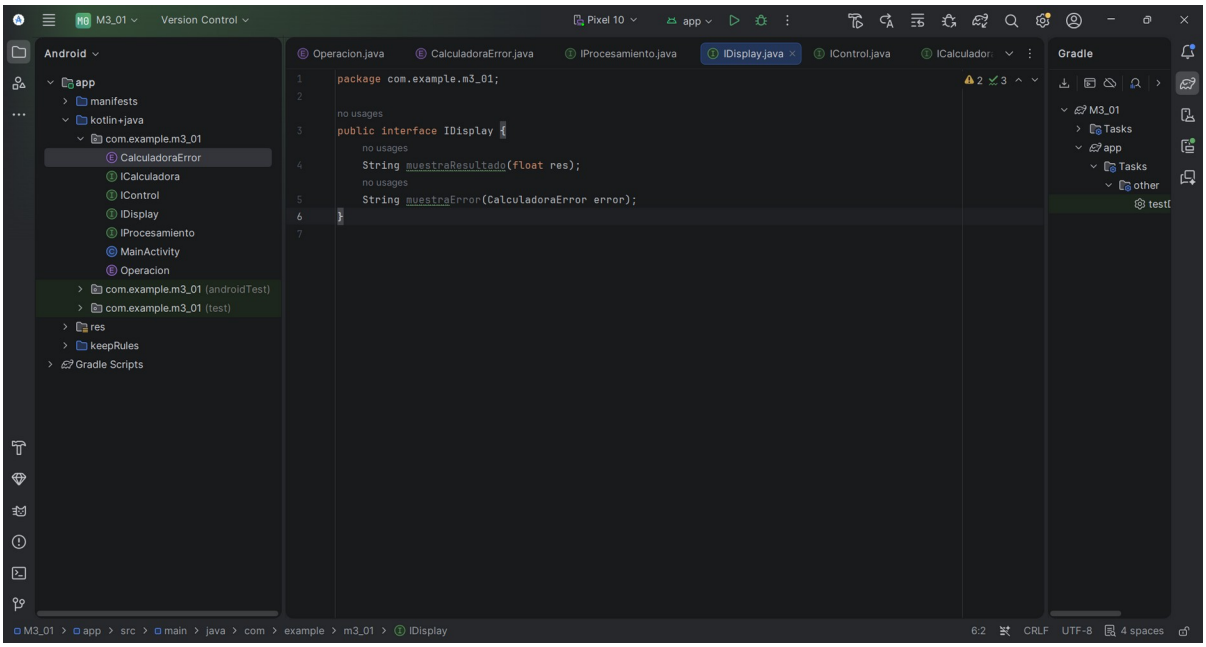


Figura 5. Interfaz IDisplay.

Interfaz IControl

```
public interface IControl {
    void introduceValorX(int x);
    void introduceValorY(int y);
    void introduceOperacion(Operacion op);
    float igual();
}
```

- **`introduceValorX / introduceValorY`**: reciben los dos números enteros con los que se va a operar. void significa que no devuelven nada.
- **`introduceOperacion(Operacion op)`**: recibe la operación a realizar, que solo puede ser una de las del enum Operacion.
- **`float igual()`**: equivale a presionar la tecla “=”: devuelve el resultado de la operación.

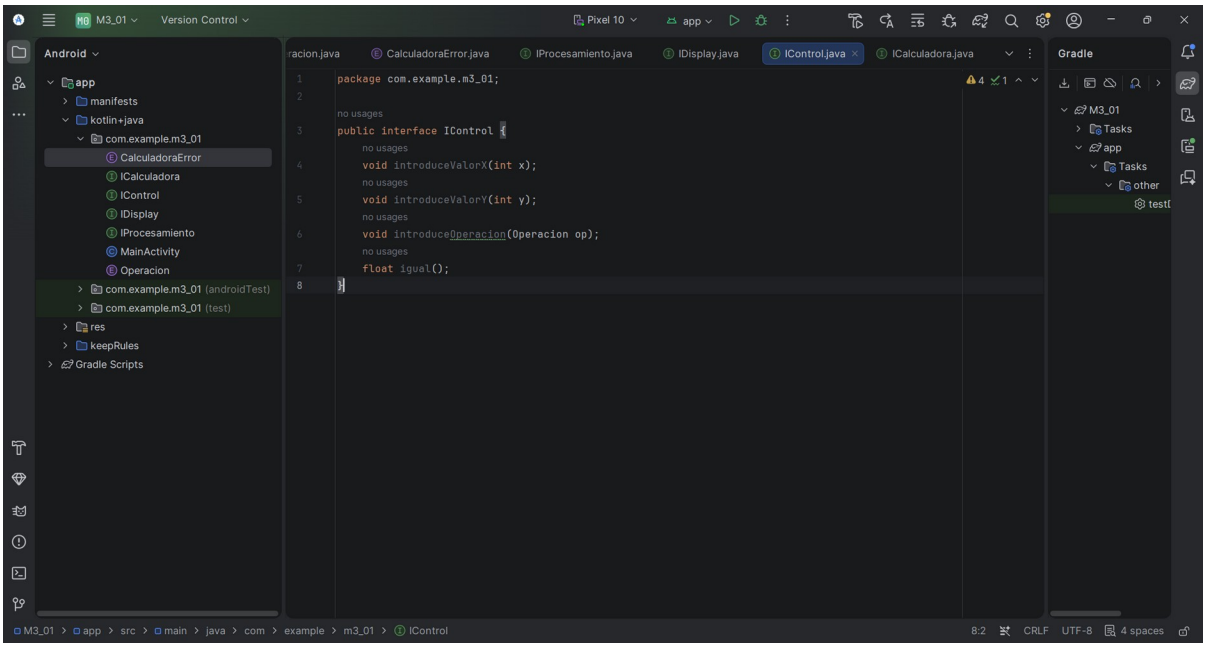


Figura 6. Interfaz IControl.

Interfaz ICalculadora

```
public interface ICalculadora {
    void setControl(IControl control);
    void setDisplay(IDisplay display);
    void setProcesamiento(IProcesamiento procesa);
}
```

- **`setControl` / `setDisplay` / `setProcesamiento`**: métodos para entregarle a la calculadora cada una de sus piezas. Reciben interfaces, no clases, por lo que cualquier pieza que cumpla el contrato puede conectarse.

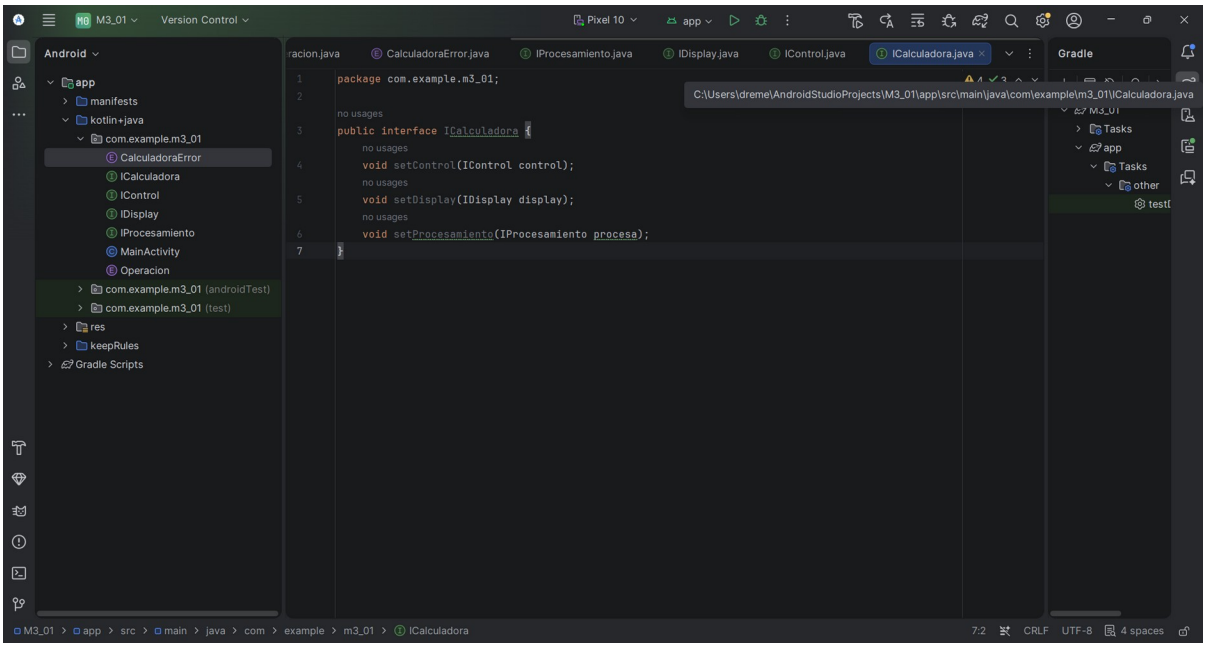


Figura 7. Interfaz ICalculadora y las seis clases base en el paquete.

4.3. Clase Procesamiento

Esta clase hace las cuentas. Es la única que sabe cómo sumar, restar, multiplicar y dividir.

```
public class Procesamiento implements IProcesamiento {

    @Override
    public float suma(float x, float y) {
        return x + y;
    }

    @Override
    public float resta(float x, float y) {
        return x - y;
    }

    @Override
    public float multi(float x, float y) {
        return x * y;
    }

    @Override
    public float div(float x, float y) {
        return x / y;
    }
}
```

- **`public class Procesamiento implements IProcesamiento`**: crea la clase y se compromete a cumplir el contrato de IProcesamiento; si faltara alguno de los cuatro métodos, Android Studio marcaría error.
- **`@Override`**: indica que cada método es el que pide la interfaz.

- **`public float suma(float x, float y)`**: método público que recibe dos números decimales y devuelve un decimal.
- **`return x + y;`**: devuelve el resultado. Los otros tres métodos funcionan igual con -, * y /.
- La división no revisa si el divisor es cero. Esto es intencional: esta clase solo calcula, y la validación se hace en otra pieza. Así cada clase tiene una sola responsabilidad.

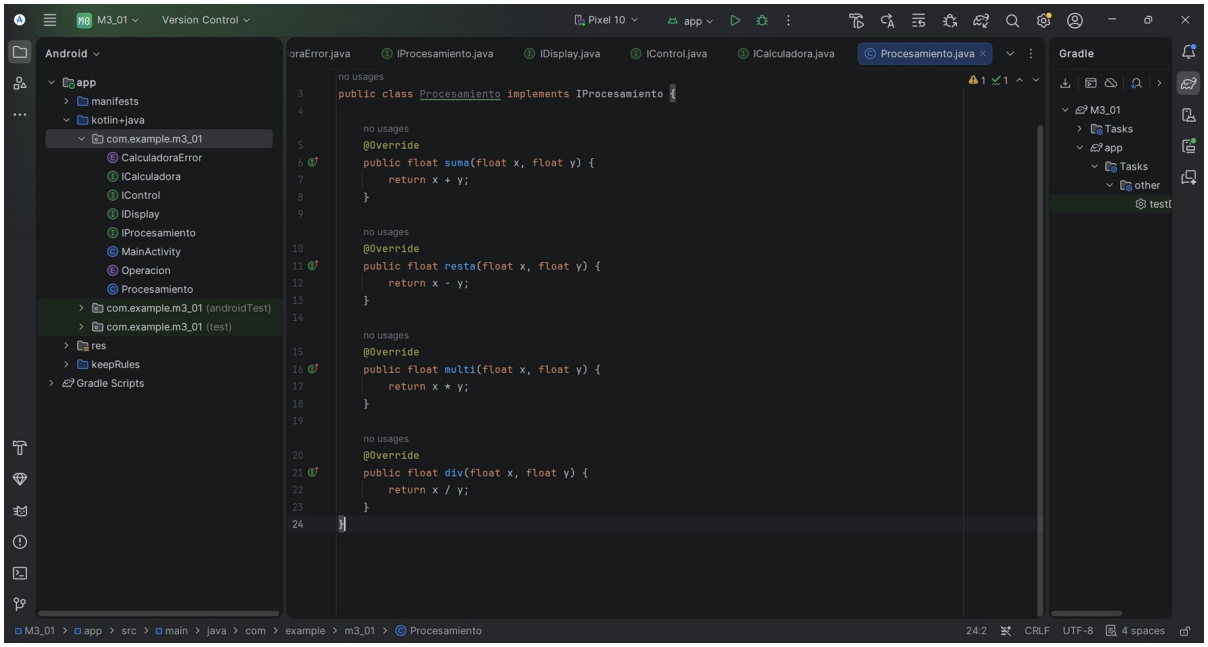


Figura 8. Clase Procesamiento.

4.4. Clase Display

Esta clase se encarga de preparar los textos que se muestran al usuario, tanto los resultados como los errores.

```
public class Display implements IDisplay {

    private static final String PREFIJO_RESULTADO = "Resultado: ";
    private static final String PREFIJO_ERROR = "Error: ";

    @Override
    public String muestraResultado(float res) {
        return PREFIJO_RESULTADO + res;
    }

    @Override
    public String muestraError(CalculadoraError error) {
        String mensaje;
        switch (error) {
            case DIV_ZERO:
                mensaje = "No se puede dividir entre cero (resultado NaN o
infinito)";
                break;
            case MAX_NUMBER:
                mensaje = "El resultado excede el número máximo permitido";
                break;
            case IMAGINARIO:
                mensaje = "El resultado es un número imaginario";
                break;
            default:
                mensaje = "Error desconocido";
        }
        return PREFIJO_ERROR + mensaje;
    }
}
```

- **`private static final String PREFIJO_RESULTADO`**: es una constante. private: solo esta clase la usa; static: pertenece a la clase y no a cada objeto; final: su valor nunca cambia; String: es un texto.
- **`return PREFIJO_RESULTADO + res;`**: une el texto “Resultado: ” con el número. Por ejemplo, con 12.0 devuelve “Resultado: 12.0”.
- **`String mensaje;`**: declara una variable donde se guardará el texto del error.
- **`switch (error)`**: revisa qué error llegó y salta al caso (case) que le corresponde.
- **`case DIV_ZERO:`**: si el error es división entre cero, asigna su mensaje.
- **`break;`**: termina el switch para que no siga revisando los demás casos.
- **`default:`**: caso de respaldo por si llegara un error no previsto.
- **`return PREFIJO_ERROR + mensaje;`**: devuelve el mensaje completo, por ejemplo “Error: No se puede dividir entre cero…”.

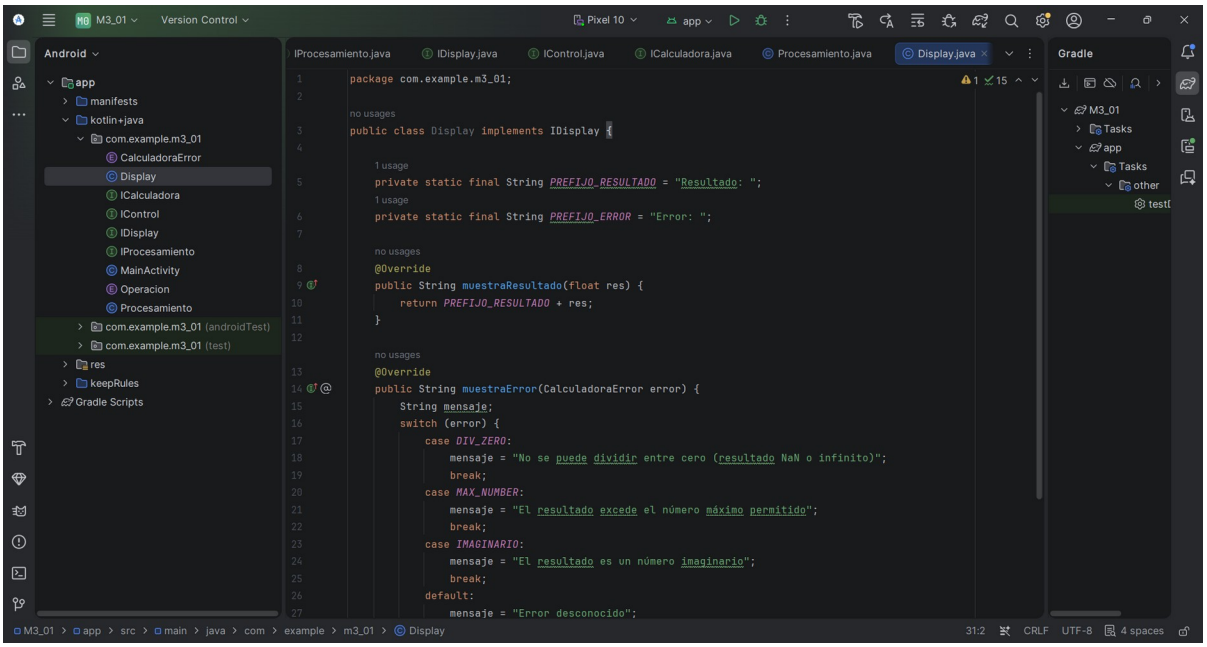


Figura 9. Clase Display: constantes y método muestraResultado.

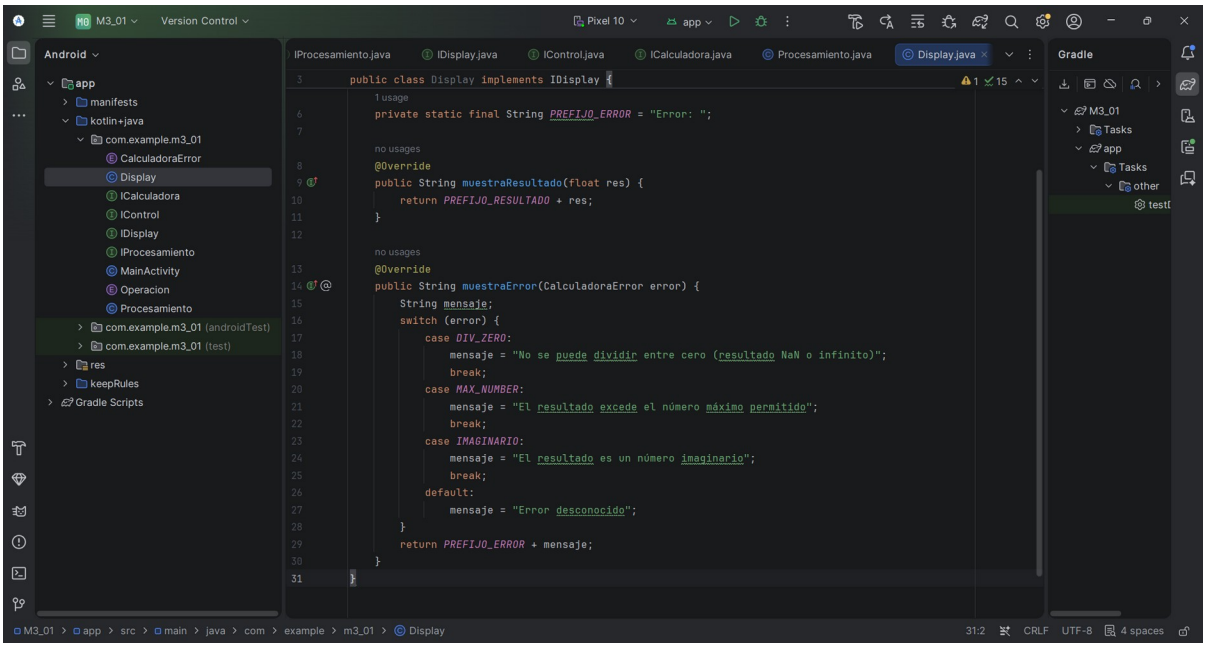


Figura 10. Clase Display: método muestraError con switch.

4.5. Clase Control

Esta clase es el “cerebro” de la calculadora: guarda los números y la operación que se eligieron y, cuando se pide el resultado, decide qué operación realizar.

```
public class Control implements IControl {

    private int valorX;
    private int valorY;
    private Operacion operacion;
    private final IProcesamiento procesamiento;

    public Control(IProcesamiento procesamiento) {
        this.procesamiento = procesamiento;
    }

    @Override
    public void introduceValorX(int x) {
        valorX = x;
    }

    @Override
    public void introduceValorY(int y) {
        valorY = y;
    }

    @Override
    public void introduceOperacion(Operacion op) {
        operacion = op;
    }

    @Override
    public float igual() {
        if (operacion == null) {
            return Float.NaN;
        }
        float x = valorX;
        float y = valorY;
        switch (operacion) {
            case SUM:
                return procesamiento.suma(x, y);
            case RES:
                return procesamiento.resta(x, y);
            case MUL:
                return procesamiento.multi(x, y);
            case DIV:
                return procesamiento.div(x, y);
            default:
                return Float.NaN;
        }
    }
}
```

- **`private int valorX; / private int valorY;`**: guardan los dos números que se introducen. Son privados para que solo esta clase pueda modificarlos (encapsulamiento).
- **`private Operacion operacion;`**: guarda la operación elegida, que debe ser una del enum.
- **`private final IProcesamiento procesamiento;`**: guarda la pieza que hace las cuentas. Es final porque, una vez asignada, no cambia. Su tipo es la interfaz IProcesamiento y no la clase Procesamiento: a Control no le importa cómo se hacen las cuentas, solo que alguien cumpla el contrato. Esto es abstracción.

- **`public Control(IProcesamiento procesamiento)`**: es el constructor. Se ejecuta al crear el objeto con new Control(...) y recibe la pieza de procesamiento.
- **`this.procesamiento = procesamiento;`**: this se refiere a la variable de la clase y la distingue del parámetro, que tiene el mismo nombre.
- **`introduceValorX / Y / Operacion`**: solo guardan el valor recibido en su variable.
- **`if (operacion == null)`**: revisa si todavía no se eligió una operación (null significa "vacío"). En ese caso devuelve Float.NaN en lugar de provocar un error.
- **`float x = valorX;`**: convierte los enteros a decimales. Sin esta conversión, 7 / 2 daría 3 en vez de 3.5, porque la división entre enteros descarta los decimales.
- **`switch (operacion)`**: según la operación, llama al método correspondiente de procesamiento y devuelve su resultado con return.

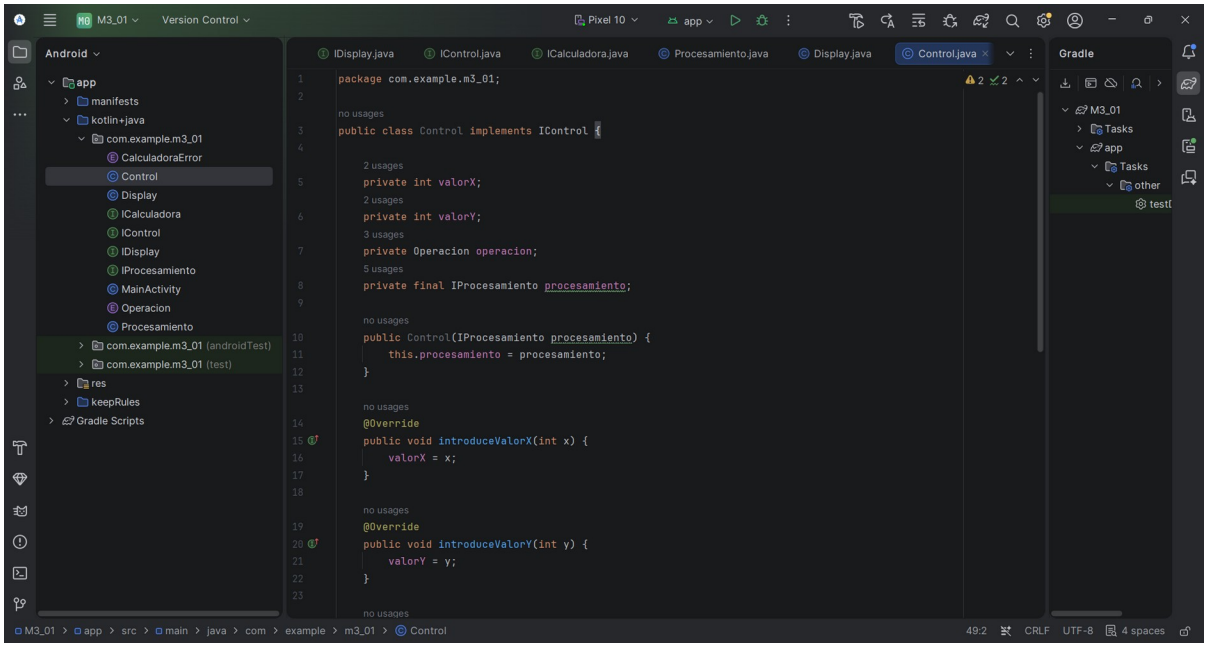


Figura 11. Clase Control: variables, constructor y métodos introduce.

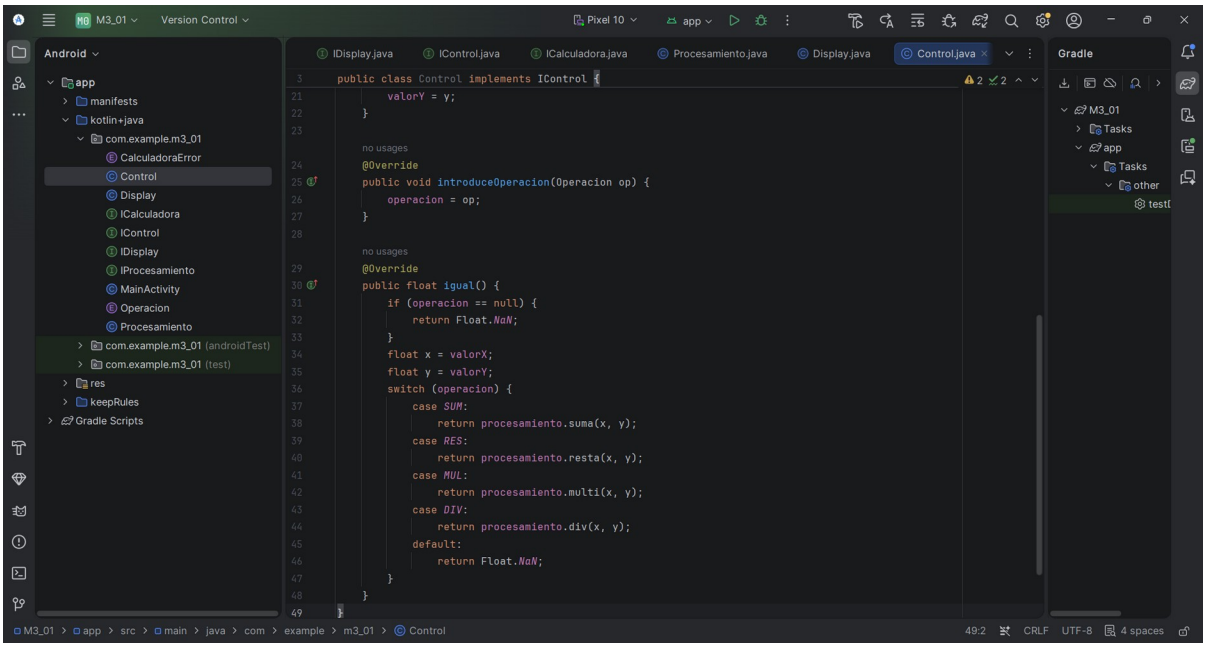


Figura 12. Clase Control: método igual con switch.

4.6. Clase Calculadora

Esta clase une las tres piezas y contiene la validación de resultados inválidos.

```
public class Calculadora implements ICalculadora {

    private IControl control;
    private IDisplay display;
    private IProcesamiento procesamiento;

    @Override
    public void setControl(IControl control) {
        this.control = control;
    }

    @Override
    public void setDisplay(IDisplay display) {
        this.display = display;
    }

    @Override
    public void setProcesamiento(IProcesamiento procesa) {
        this.procesamiento = procesa;
    }

    public String calcular(int x, Operacion op, int y) {
        control.introduceValorX(x);
        control.introduceValorY(y);
        control.introduceOperacion(op);

        float resultado = control.igual();

        if (Float.isNaN(resultado) || Float.isInfinite(resultado)) {
            if (op == Operacion.DIV) {
                return display.muestraError(CalculadoraError.DIV_ZERO);
            }
            return display.muestraError(CalculadoraError.MAX_NUMBER);
        }
        return display.muestraResultado(resultado);
    }
}
```

- **`private IControl control; (y las otras dos)`**: guardan las tres piezas, declaradas con su interfaz para que cualquier implementación pueda conectarse.
- **`setControl / setDisplay / setProcesamiento`**: reciben cada pieza desde fuera y la guardan.
- **`public String calcular(int x, Operacion op, int y)`**: método que se usa desde MainActivity. Recibe los dos números y la operación, y devuelve el texto final.
- **`control.introduceValorX(x); …`**: entrega los datos a Control.
- **`float resultado = control.igual();`**: pide el resultado, como si se presionara “=”.
- **`Float.isNaN(resultado)`**: devuelve verdadero si el resultado “no es un número”, como ocurre con 0 ÷ 0.
- **`Float.isInfinite(resultado)`**: devuelve verdadero si el resultado es infinito, como ocurre con 5 ÷ 0.
- **`||`**: significa “o”: basta con que se cumpla una de las dos condiciones.

- **`if (op == Operacion.DIV)`**: si el resultado inválido vino de una división, se muestra el error DIV_ZERO; si vino de otra operación, solo puede deberse a un número demasiado grande, y se muestra MAX_NUMBER.
- **`return display.muestraResultado(resultado);`**: si el resultado es válido, Display arma el texto “Resultado: …”.
- En Java, dividir un float entre cero no detiene el programa con un error, sino que produce Infinity o NaN. Por eso es necesario validarlo de forma explícita.

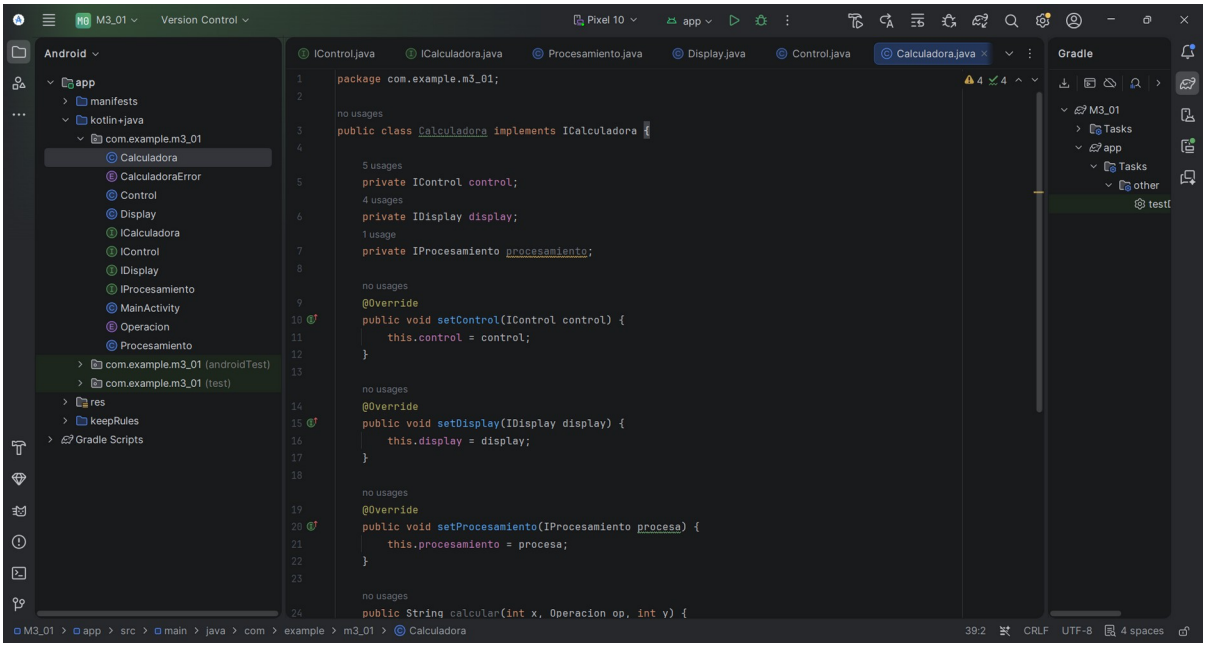


Figura 13. Clase Calculadora: variables y métodos set.

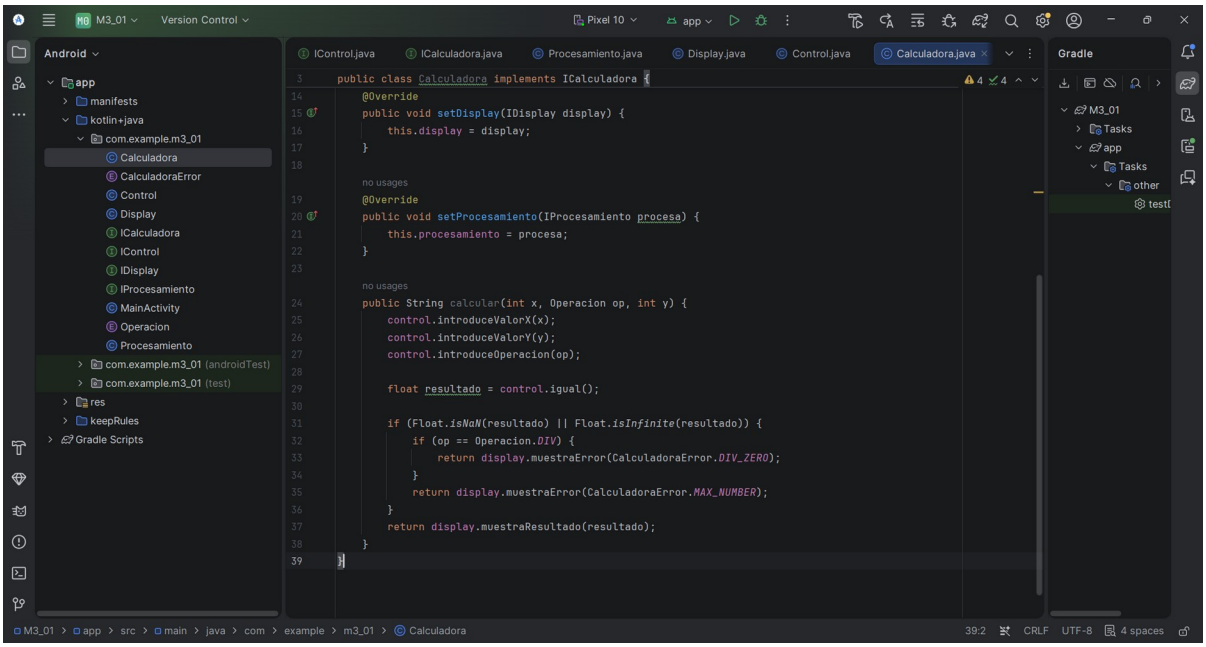


Figura 14. Clase Calculadora: método calcular con la validación de NaN.

4.7. Pruebas en MainActivity

Dentro del método onCreate() de MainActivity se crearon las piezas, se armó la calculadora y se ejecutaron siete pruebas.

```
// 1. Crear las piezas
Procesamiento procesamiento = new Procesamiento();
Control control = new Control(procesamiento);
Display display = new Display();

// 2. Armar la calculadora
Calculadora calculadora = new Calculadora();
calculadora.setProcesamiento(procesamiento);
calculadora.setControl(control);
calculadora.setDisplay(display);

// 3. Pruebas
Log.d("M3_01", "8 + 4 -> " + calculadora.calcular(8, Operacion.SUM, 4));
Log.d("M3_01", "8 - 4 -> " + calculadora.calcular(8, Operacion.RES, 4));
Log.d("M3_01", "8 * 4 -> " + calculadora.calcular(8, Operacion.MUL, 4));
Log.d("M3_01", "8 / 4 -> " + calculadora.calcular(8, Operacion.DIV, 4));
Log.d("M3_01", "7 / 2 -> " + calculadora.calcular(7, Operacion.DIV, 2));
Log.d("M3_01", "5 / 0 -> " + calculadora.calcular(5, Operacion.DIV, 0));
Log.d("M3_01", "0 / 0 -> " + calculadora.calcular(0, Operacion.DIV, 0));
```

- `// …`: las líneas que empiezan con dos diagonales son comentarios; Java las ignora y sirven para explicar el código.
- **`new Procesamiento()`**: new crea un objeto a partir de una clase.
- **`new Control(procesamiento)`**: al crear Control se le entrega el objeto procesamiento, porque su constructor lo pide.
- **`calculadora.setProcesamiento(…) …`**: conecta las tres piezas a la calculadora.

- **`calculadora.calcular(8, Operacion.SUM, 4)`**: realiza la operación y devuelve el texto del resultado.
- **`Log.d("M3_01", …)`**: imprime en Logcat el texto, con la etiqueta M3_01 para poder filtrarlo.
- Las pruebas cubren las cuatro operaciones, una división con resultado decimal (7 ÷ 2) y los dos casos problemáticos: 5 ÷ 0, que produce infinito, y 0 ÷ 0, que produce NaN.

4.8. Resultados en Logcat

La aplicación se ejecutó en el emulador Pixel 10 (Android 17, API 37). Como la calculadora no tiene interfaz gráfica, la pantalla conserva el texto de la plantilla y los resultados se consultan en Logcat con el filtro tag:M3_01.

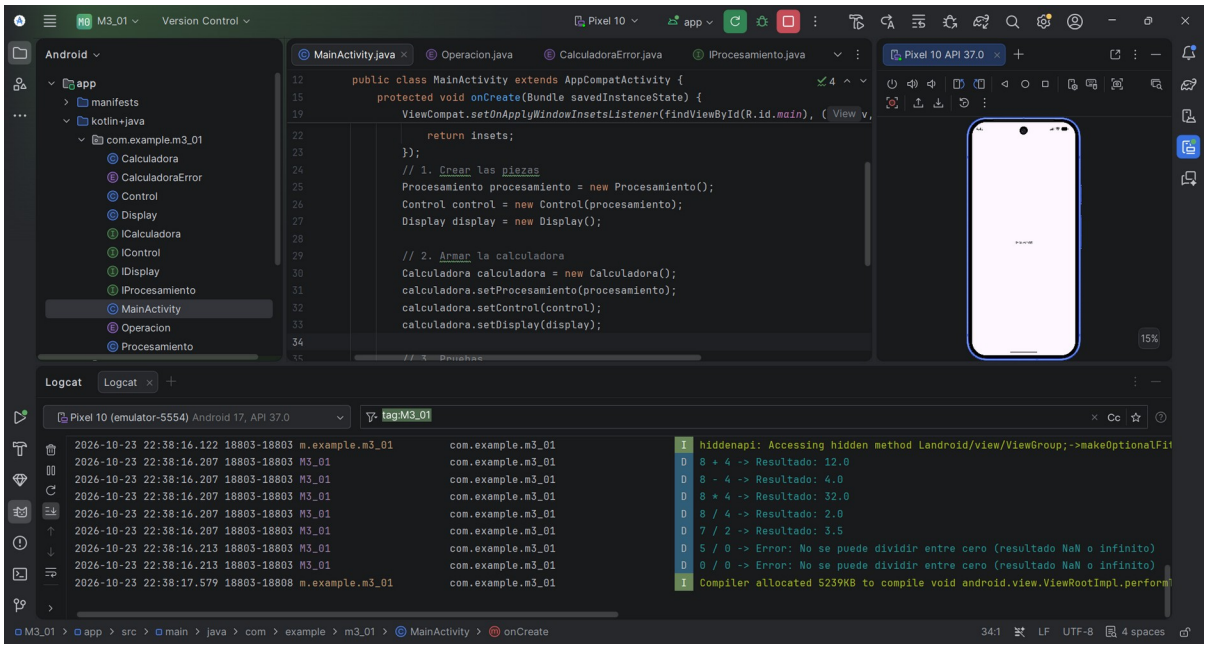


Figura 15. Código de las pruebas y resultados en Logcat.

Descripción: Logcat muestra los siete resultados esperados: 8 + 4 = 12.0, 8 − 4 = 4.0, 8 × 4 = 32.0, 8 ÷ 4 = 2.0 y 7 ÷ 2 = 3.5, este último con decimales gracias a la conversión a float. En las divisiones 5 ÷ 0 y 0 ÷ 0 se muestra el mensaje “Error: No se puede dividir entre cero (resultado NaN o infinito)”, lo que confirma que la validación funciona y que la aplicación no se detiene. Las líneas marcadas con I son mensajes del sistema que coinciden con el nombre del paquete.

Código fuente de esta actividad: https://github.com/Mr-X22/android-2027-1/tree/main/M3-01

5. Conclusión

Siendo sincero, esta actividad se me hizo un poco menos interesante que las demás, y creo que fue por lo confusa que me resultó. Había tantas clases que me confundía entre ellas y me costaba

entender cómo se conectaban. Nunca había hecho una calculadora en ese formato; solo la había representado en diagramas de flujo, y verla funcionando de verdad fue distinto.

Me desconcertó ver que, al dividir entre cero, el programa no se detenía sino que mostraba NaN o infinito. Después entendí que eso hay que validarlo con Float.isNaN() y Float.isInfinite(), para mostrar un mensaje claro en lugar de un resultado incorrecto.

Aun así, me quedo con una idea importante: separar el programa en clases hace que cada parte se encargue de una sola función y que el trabajo se haga por fases, como si una clase recibiera los datos (Control), otra los procesara (Procesamiento) y otra mostrara el resultado (Display). Así cada parte es más fácil de entender y revisar, y si quiero cambiar algo, como la forma de mostrar los resultados, no tengo que tocar lo demás.

6. Referencias

BitBoss. (2021, 30 de agosto). *Programación orientada a objetos (POO): Abstracción, encapsulamiento, herencia, polimorfismo* [Video]. YouTube. https://www.youtube.com/watch?v=SI7O81GMG2A

Blasco, J. L. (2023, 22 de noviembre). *Introducción a POO en Java: Interfaces y paquetes*. OpenWebinars. https://openwebinars.net/blog/introduccion-a-poo-en-java-interfaces-y-paquetes/

Charly Cimino. (2022, 22 de junio). *INTERFACES (implements) en Java. Acá lo vas a entender* [Video]. YouTube. https://www.youtube.com/watch?v=aKf8ivfyxnI

Google. (s. f.). *View logs with Logcat*. Android Developers. https://developer.android.com/studio/debug/logcat

LearnWtutorials. (2016, 27 de abril). *Tutorial 6 de Java - Constantes (static final)* [Video]. YouTube. https://www.youtube.com/watch?v=W52wGw1Ugns

Oracle. (s. f.-a). *Enum types*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/enum.html

Oracle. (s. f.-b). *Float (Java SE 17 & JDK 17)*. https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Float.html

Oracle. (s. f.-c). *Interfaces*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html

ProgramarYa - By JuanDMeGon. (2020, 21 de julio). *Atributos constantes final en Java* [Video]. YouTube. https://www.youtube.com/watch?v=cGl3kVxt9rY

TodoCode. (2021, 24 de octubre). *Métodos constructores y objetos en Java | POO con Java | Explicación fácil #4* [Video]. YouTube. https://www.youtube.com/watch?v=fdfXkiuVHp4

TodoCode. (2024a, 28 de enero). *ENUMS en Java | Explicación fácil* [Video]. YouTube. https://www.youtube.com/watch?v=khY9QhgL5S8

TodoCode. (2024b, 14 de febrero). *Métodos de ENUMS en Java | Explicación fácil* [Video]. YouTube. https://www.youtube.com/watch?v=FzCPx6sIctQ

William Vargas. (2024, 8 de marzo). *12. POO: abstracción, encapsulamiento, herencia, polimorfismo* [Video]. YouTube. https://www.youtube.com/watch?v=da0TiomiQNM